

MindNav Experiment Record

1 Purpose

This document records the experimental path behind the MindNav paper. It is an internal reference, not manuscript text. The goal is to make clear what was run, why the next experiment was introduced, and what each stage showed.

The experiments proceeded in three stages:

1. HM3D VAL_MINI 30-episode sanity checks.
2. The original MindNav 50-episode evaluation on HM3D VAL.
3. KG-complexity analysis, KG fixes, and a 50-episode heuristic control.

Unless otherwise stated, SR denotes success rate and SPL denotes success-weighted path length. All multi-robot runs use two robots. Qwen2.5-7B denotes Qwen2.5-7B-Instruct deployed locally. All DeepSeek runs use DeepSeek-V4-Flash through an API; there is no local DeepSeek deployment in these experiments.

2 Stage 1: HM3D VAL_MINI 30-Episode Sanity Checks

The first stage was a small-split sanity check. The purpose was to verify that the KG tool-calling pipeline could run end-to-end, compare it with Co-NavGPT under similar infrastructure, and check whether the LLM backend made a visible difference on a 30-episode split.

At this stage, MindNav KG tool-calling was promising: it outperformed Co-NavGPT on both Qwen and DeepSeek-V4-Flash API backends, and DeepSeek-V4-Flash was slightly better than Qwen on this small split. However, the split is too small to support a strong model comparison. These runs mainly established that the KG pipeline was worth evaluating on a larger fixed episode set.

3 Stage 2: Original MindNav 50-Episode Evaluation

The second stage evaluated MindNav KG and Co-NavGPT on a matched 50-episode HM3D VAL matrix. This was the first more reliable comparison between the KG tool-calling interface and the flat-text Co-NavGPT interface.

MindNav KG improved over Co-NavGPT under both matched backends. With Qwen, the gain was +0.020 SR and +0.072 SPL. With DeepSeek-V4-Flash API, the gain was +0.040 SR and +0.006 SPL. The strongest SPL in this matrix was MindNav KG with Qwen.

The Qwen–DeepSeek comparison was less straightforward. Both MindNav KG backends achieved the same SR of 0.700, and Qwen had slightly higher SPL. At the episode level, 29 episodes succeeded under both models, 9 failed under both models, 6 succeeded only under Qwen, and 6 succeeded

Table 1: Stage 1: HM3D VAL_MINI sanity checks.

Method	Backend / variant	Episodes	Successes	SR	SPL
MindNav KG tool-calling	Qwen2.5-7B	30	22	0.733	0.344
MindNav KG tool-calling	DeepSeek-V4-Flash API	30	23	0.767	0.379
Co-NavGPT	Qwen2.5-7B	30	20	0.667	0.332
Co-NavGPT	DeepSeek-V4-Flash API	30	19	0.633	0.317
MCoCoNav	Qwen2.5-VL-7B	30	21	0.700	0.396

Table 2: Stage 2: Original 50-episode HM3D VAL matrix.

Method	Backend	Episodes	Successes	SR	SPL
MindNav KG	Qwen2.5-7B	50	35	0.700	0.444
MindNav KG	DeepSeek-V4-Flash API	50	35	0.700	0.435
Co-NavGPT	Qwen2.5-7B	50	34	0.680	0.372
Co-NavGPT	DeepSeek-V4-Flash API	50	33	0.660	0.429

only under DeepSeek. Thus, the aggregate success rate was tied, but the two models had different decision behavior.

The working interpretation after this stage was that raw language-model strength was not the only limiting factor. The LLM interface was narrow: the model received active frontiers, room-type hints, nearby objects, target priors, serialized KG text, recent history, and tool results, and it only had to assign robots to frontiers. In this setting, stronger general reasoning may not directly translate into higher SR/SPL if the final task is mostly stable ranking and assignment under noisy perception.

4 Stage 3: KG Complexity, KG Fixes, and Heuristic Control

The third stage was motivated by the Stage 2 observation that DeepSeek did not clearly outperform Qwen. We therefore inspected whether the KG itself was too complex or noisy, changed the KG update procedure, and added a deterministic heuristic policy as a control. The goal was not to introduce a new final method, but to understand whether performance was limited by KG construction, LLM frontier choice, or low-level navigation dynamics.

4.1 Why We Suspected KG Complexity and Noise

The earlier KG could mislead frontier assignment in several ways:

- **Stale frontier state.** Frontier indices are step-local, but older room nodes could retain previous frontier metadata.
- **Stale inferred topology.** Edges such as `connected_to`, `connected_via`, `next_to`, and `near` could persist after the map changed.
- **Object duplication.** The same object category could be inserted multiple times from frontier priors and semantic detections.
- **Weak confidence calibration.** Some object nodes used default confidence rather than confidence estimated from the semantic-map patch.
- **Weak wall/connectivity consistency.** Nearby room-grid cells could be connected even when wall evidence suggested separation.

- **Graph clutter.** Auxiliary object-to-room-type hints could add many extra edges and make the serialized KG harder to inspect.

These issues are especially relevant for a model that reads the KG structure more aggressively. A model that mostly follows target priors may be insensitive to KG noise; a model that tries to use topology, explored state, path hints, and object–room relations can be helped by a clean KG but hurt by a noisy one.

4.2 What We Changed in the KG

The KG update was modified to make the graph more current and conservative:

- clear `active_frontier` markers at each global step and only mark currently extracted frontiers as active;
- rebuild inferred room topology from current evidence instead of accumulating stale room-connectivity edges;
- remove direct room connections that conflict with wall-separation evidence;
- estimate object confidence from the semantic-map patch and cap it at 0.99;
- merge repeated same-category object detections with noisy-OR certainty,

$$c_{\text{new}} = 1 - (1 - c_{\text{old}})(1 - c_{\text{det}}),$$

and deduplicate nearby object nodes after graph construction;

- store room-type suggestions as object properties rather than expanding them into many auxiliary edges;
- add a direct `TARGET_<category>` node when the target category is visible with sufficient semantic-map evidence.

The intended effect was a cleaner decision state: fewer duplicate objects, fewer stale inferred edges, more conservative topology, and active frontier ids that matched the current planning step.

4.3 Heuristic Policy Added as a Control

We also added a deterministic KG heuristic. It uses the same optimized KG, frontier enrichment, target-prior signal, and target shortcut as MindNav, but it does not call an LLM. This made it possible to ask whether an explicit scoring rule could match or outperform LLM frontier assignment.

If the target is directly represented by a `TARGET_<target>` node, or if an object of the target category has certainty above $\tau = 0.5$, the heuristic sends the nearest robot to the frontier closest to the target. Otherwise, each robot–frontier pair is scored as

$$s(r, f) = 4.00 p_{\text{target}}(f) + 0.25 \min(c_{\text{room}}(f), 1) + 0.15 \min(A(f)/80, 1) \\ - 0.35 \min(d(r, f)/500, 1) - 0.45 n_{\text{recent}}(f),$$

where p_{target} is the frontier target prior, c_{room} is room confidence, A is frontier area, $d(r, f)$ is robot–frontier distance, and n_{recent} counts recent repeated selections. For two robots, all distinct frontier pairs are enumerated, with an additional diversity bonus for larger inter-frontier distance.

Table 3: Stage 3: 50-episode reruns using the optimized MindNav KG pipeline.

Pipeline	Navigation brain / policy	Episodes	Successes	SR	SPL
MindNav optimized KG	Qwen2.5-7B LLM	50	35	0.700	0.444
MindNav optimized KG	DeepSeek-V4-Flash API	50	32	0.640	0.384
MindNav optimized KG	Deterministic KG heuristic	50	36	0.720	0.422

Table 4: Decision behavior on the Stage 3 val50 runs.

Variant	Steps	Target	Target rate	Same F	Top prior	Avg chosen	Avg max
KG heuristic	386	76	0.197	0.039	0.969	0.396	0.483
Qwen2.5-7B LLM	352	67	0.190	0.028	0.972	0.427	0.527
DeepSeek-V4-Flash API	409	54	0.132	0.029	0.895	0.388	0.498

4.4 Stage 3 Results

All three Stage 3 runs used the optimized MindNav KG pipeline. They differed only in the navigation brain: Qwen2.5-7B, DeepSeek-V4-Flash API, or the deterministic KG heuristic. With this optimized KG pipeline, Qwen reproduced the Stage 2 Qwen result exactly, with $SR = 0.700$ and $SPL = 0.444$. DeepSeek-V4-Flash dropped to $SR = 0.640$ and $SPL = 0.384$. The heuristic policy achieved the highest SR among the Stage 3 runs, $SR = 0.720$, although its SPL was lower than Qwen’s SPL.

The main lesson from Stage 3 is that this setup is not a pure “larger LLM is better” benchmark. A conservative policy that closely follows target prior, room evidence, distance, recency, and frontier diversity is very competitive. Qwen behaved closer to this conservative pattern, whereas DeepSeek appeared more willing to deviate from the highest target-prior frontier after the KG changes.

5 Cross-Stage Interpretation

Across the three stages, the experimental story is:

- **Stage 1** showed that KG tool-calling was viable on VAL_MINI and generally better than Co-NavGPT on the same small split.
- **Stage 2** showed that MindNav KG improved over Co-NavGPT on a 50-episode matrix, but also showed that DeepSeek did not clearly outperform Qwen2.5-7B.
- **Stage 3** showed that after KG cleanup, Qwen remained stable, DeepSeek became worse, and a deterministic heuristic achieved the best SR.

The likely bottleneck is therefore the embodied frontier-control interface, not only language-model capacity. The LLM receives an already compressed state and outputs a small assignment decision. Low-level navigation, perception quality, target-prior calibration, KG noise, and post-processing all affect final SR/SPL. In this regime, a model or policy that reliably follows the strongest numeric evidence can outperform a model that explores more nuanced but noisy KG structure.

6 Detailed Decision Diagnostics

The trace statistics support the Stage 3 interpretation. DeepSeek did not fail because it ignored tools; it made more tool calls overall, partly because its episodes lasted longer. The key behavioral difference was frontier selection.

Table 5: KG tool-call counts on the Stage 3 LLM runs.

Variant	Query objects	Estimate prob.	Assign	Check target
Qwen2.5-7B LLM	782	782	284	67
DeepSeek-V4-Flash API	1028	1028	354	54

Qwen chose a frontier with the highest target prior in 97.2% of decision steps, close to the heuristic’s 96.9%. DeepSeek did so in only 89.5% of decision steps. DeepSeek also had fewer target-found shortcut steps, more total decision steps, and a lower average chosen prior. Same-frontier assignment rates were low for all variants, so duplicate assignment of both robots to the same frontier was not the primary failure mode.

These diagnostics suggest a concrete next direction: let the LLM estimate room or frontier probabilities, but make the final frontier assignment deterministic, with explicit distance, diversity, recency, and validity terms.